

Wrench Transport

Retained contact impulses and coupled body response
for rigid-body simulation

Joshuah Rainstar

3 October 2026

Abstract

Wrench Transport is a rigid-body engine for tight contact and persistent support at real-time computational cost. In three common 64-body packing scenes, its 60 Hz configuration settles every scene with a maximum final overlap of 0.0136 mm, using a median 0.336 s of compute for eight simulated seconds. Default Rapier settles all three at 0.675 times that cost, with 513 times the maximum overlap. A higher-rate Rapier configuration costs 1.94 times as much, retains 466 times the overlap, and settles two scenes. The tested MuJoCo configurations require 19–49 times Wrench Transport’s compute; even the one with the smallest overlap retains 17.7 times as much and does not meet the settling criterion. Wrench Transport achieves this operating point by retaining contact impulses, scaling regularization to supported load, and solving the coupled body response through sparse block elimination. We describe the C++20 engine, its convex inner contact problem, and its mechanical and numerical checks. The comparisons use common geometry, an independent overlap calculation, and sleeping disabled throughout.

1 Introduction

Tight geometric contact and sustained rest must be affordable within a simulation frame. Wrench Transport couples these objectives: a 60 Hz step resolves the connected bodies’ response while preserving the impulses that already support them. In the packing experiments, this gives much smaller final overlap than Rapier at compute costs of the same order, and smaller overlap than MuJoCo at a fraction of the tested configurations’ cost.

The motivating intuition treats contact as transport. An influence enters the contact network, its consequences pass through the connected bodies, and the resulting field must support a consistent motion. The useful unit of that exchange is a *wrench*: the force and torque applied to one rigid body,

$$\mathcal{W} = \begin{bmatrix} F \\ \tau \end{bmatrix}, \quad F = \sum_c f_c, \quad \tau = \sum_c (p_c - x) \times f_c. \quad (1)$$

Its dual variable is the body’s twist, consisting of linear and angular velocity. Eliminating a body’s unknown twist condenses both its load and its response onto the remaining contact boundary. The supporting bodies therefore respond to the structure they carry, including its rotational coupling.

Two properties guide the engine. First, a static load must persist: numerical transport cannot attenuate the impulse that holds a body at rest. Second, a contact’s numerical scale should reflect the mass it supports, which may greatly exceed the mass of either adjacent body. Retained impulses address the first property, while carried-mass and previous-load scaling address the second.

The engine combines these choices with a velocity-space convex inner solve. Related primal contact formulations include the Semi-Analytic Primal solver (SAP) of Castro, Permenter, and Han [1]. Wrench Transport uses a retained-impulse outer update and a load-dependent regularization, with a sparse six-dimensional body-block factorization for each Newton system.

2 One contact field, one coupled response

Let $V \in \mathbb{R}^{6n}$ collect the awake bodies' twists, V^* denote their free-motion twists after external forces, and $M \succ 0$ their block mass/inertia matrix. For contact c , $J_c V$ is relative contact velocity; λ_c is its retained impulse; $R_c > 0$ is a scalar algorithmic regularization; and $\hat{v}_c$ is a target velocity held fixed during an inner solve. The friction cone is

$$K_c = \{(\gamma_t, \gamma_n) : \gamma_n \geq 0, \|\gamma_t\| \leq \mu_c \gamma_n\}. \quad (2)$$

Writing P_{K_c} for Euclidean projection onto that cone, the contact portion of an inner problem is

$$y_c(V) = \lambda_c - \frac{J_c V - \hat{v}_c}{R_c}, \quad \gamma_c(V) = P_{K_c}(y_c(V)), \quad (3)$$

$$\Phi(V) = \frac{1}{2}(V - V^*)^T M (V - V^*) + \sum_c \frac{R_c}{2} \|P_{K_c}(y_c(V))\|^2. \quad (4)$$

Its gradient expresses momentum balance directly:

$$\nabla \Phi = M(V - V^*) - \sum_c J_c^T \gamma_c. \quad (5)$$

On a smooth projection branch, the Newton matrix is

$$H = M + \sum_c J_c^T G_c J_c, \quad G_c = R_c^{-1} D P_{K_c}(y_c) \succeq 0. \quad (6)$$

Thus each fixed inner problem is strongly convex. The implementation uses branch derivatives at contact transitions, a line search, and sparse 6×6 block elimination. Additional bounded rows implement rolling resistance and the hold; Equation (4) describes the contact core.

2.1 What the macro-frame becomes

Partition a Newton system into a body to eliminate and its remaining boundary:

$$\begin{bmatrix} A & B \\ B^T & D \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} a \\ b \end{bmatrix} \implies (D - B^T A^{-1} B)v = b - B^T A^{-1} a. \quad (7)$$

The Schur complement transports both load and response. Back-substitution then returns the boundary's response to the eliminated body. This is the precise realization of “everything above contributes now.” It is standard elimination, applied to the contact graph; the implementation stores block transfers and uses a minimum-degree ordering.

A body contributes six degrees of freedom; a condensed cluster retains the coupled degrees of freedom on its boundary. The cost depends on graph fill and elimination order. This use of structured body response connects to classical constrained-dynamics algorithms [2].

2.2 Retention and carried load

After an inner solve, retained rows set $\lambda_c \leftarrow \gamma_c$. The next pass also updates the sliding normal target with a $-\mu_c \|v_{t,c}\|$ shift. Regularization is scaled by the largest of local effective mass, an estimated carried-mass share, and mass inferred from last frame's normal impulse $\lambda_n / (\|g\| h)$. This makes the numerical scale responsive to the load a contact actually supports.

At a consistent resting state, $V = 0$, targets and retained impulses agree, and the gradient vanishes: support can be a fixed point without deliberately creeping through a soft spring. The implementation terminates the outer iteration on retained-impulse and sliding-shift changes, with a maximum of six passes. Inner residual tolerances tighten as the retained field converges.

3 Engine architecture

The standalone implementation is a C++20 static library. The public interface provides shape cooking, dynamic and static body creation, pose and velocity queries, fixed-step simulation, contact inspection, raycasting, a bounded crane hold, and rest-state queries. Simulation code has no renderer or window-system dependency.

Geometry and collision. Input point clouds are cooked into convex hulls with polygonal faces, edge adjacency, mass, center of mass, and inertia. A body may contain multiple convex hulls to represent compound geometry. The narrow phase uses face and edge separating axes, Gauss-map edge pruning, and clipped contact manifolds checked against supporting witnesses. A speculative margin includes the bodies’ estimated motion. Cached separated pairs can be rejected while a conservative movement bound remains smaller than their separation; sufficiently stationary touching pairs reuse their local manifold.

Contact and integration. Each active body’s free twist includes gravity and configured damping. Contact rows contribute friction-cone impulses; separate bounded rows model rolling resistance and the crane hold. Restitution is activated at contact closure above a speed threshold. After the coupled solve, the engine integrates position and orientation and retains the resulting contact impulses for the next frame.

Rest and wake. Contacts connect bodies into islands. An island sleeps when every free body remains below the linear and angular thresholds for the required quiet interval. Contact from an awake body, removal of supporting geometry, or an explicit pose change wakes the relevant bodies. The performance comparisons disable sleeping to expose the cost and stability of continued contact simulation.

Computational structure. A body contributes a 6×6 inertia block, and a contact couples at most two body blocks. Symbolic ordering is reused when the contact graph is unchanged. Within a frame, a numerical factor can be reused while the relevant contact branches remain unchanged; sliding branches generally require refactorization. Floating-point contraction is disabled in the measured build.

4 Experimental design

The comparison scene contains 64 dynamic convex bodies, drawn from eight shape classes: boxes, sphere approximations, cylinders, cones, thin sheets, prisms, general polyhedra, and irregular rocks. The bodies begin on a four-layer grid with seeded orientations inside a 0.37×0.37 m container. Every engine receives the same hull vertices, initial poses, density of 2600 kg/m^3 , friction coefficient of 0.5, and gravity of 9.81 m/s^2 . Restitution, damping, and rolling resistance are disabled for this workload.

Each run advances eight simulated seconds and samples motion at 60 Hz. A body-speed measure combines translation and rotation as $\|v\| + r \|\omega\|$, where r is its hull radius. Settling is the first sample after which the maximum body speed stays below 4 mm/s through the end of the run. Final interpenetration is evaluated with one independent exhaustive separating-axis calculation on the exported hull poses, including body pairs, walls, and floor.

Wrench Transport runs at 60 and 120 Hz. Rapier 0.21.0 is evaluated at 60 Hz with four solver iterations, both its default length unit and a 0.05 m length unit, and at 240 Hz with eight iterations and the smaller length unit. MuJoCo 3.2.3 uses Newton solving, elliptic friction cones, and multi-contact convex collision at 60 and 120 Hz, plus a 480 Hz configuration with a 0.005 s contact time constant. These configurations expose the cost–accuracy tradeoff rather than equating iteration counts across different solvers.

5 Evaluation

At 60 Hz, Wrench Transport settles all three scenes in 1.20–1.37 simulated seconds, with a maximum final overlap of 0.0136 mm. Its median compute cost is 0.336 s per eight simulated seconds, or 23.8 times real time. The result is an accuracy–cost advantage: none of the tested alternatives reaches this overlap, including configurations with substantially greater computational cost.

The cross-engine comparison uses seeds 1, 7, and 19, one warm-up and three measured repetitions per configuration, with rotating execution order. The hardware is an Apple M4 Mac mini running macOS 26.5. Wrench Transport uses an AppleClang 21 Release build; MuJoCo runs its native solver through Python; Rapier runs its WebAssembly distribution through Node. Timings include the respective step-call boundary. Per-run records retain runtime, settling time, final overlap, and remaining motion.

Configuration	Compute (s)	Cost ratio	Overlap (mm)	Settled
Wrench Transport, 60 Hz	0.336	1.00	0.0136	3/3
Wrench Transport, 120 Hz	0.481	1.43	0.0244	3/3
Rapier, 60 Hz	0.227	0.67	6.975	3/3
Rapier, 60 Hz, scaled	0.181	0.54	9.794	0/3
Rapier, 240 Hz, scaled	0.653	1.94	6.330	2/3
MuJoCo, 60 Hz	6.291	18.72	30.637	0/3
MuJoCo, 120 Hz	10.075	29.98	6.064	0/3
MuJoCo, 480 Hz, stiff	16.400	48.80	0.241	0/3

Compute cost is the median of nine measured runs, each advancing eight simulated seconds; the ratio uses Wrench Transport at 60 Hz as its denominator. Overlap is the largest final penetration across all runs. Settled counts scenes satisfying the sustained 4 mm/s criterion. Three bodies leave the container in the worst MuJoCo 60 Hz case; all other configurations retain every body.

Rapier: similar cost, much larger overlap. Default 60 Hz Rapier costs 0.675 times as much and settles every scene, with 513 times the maximum overlap. Reducing its length unit lowers cost further but leaves every scene moving. At 240 Hz with eight iterations, cost rises to 1.94 times Wrench Transport’s, maximum overlap remains 466 times larger, and two scenes settle. These are ratios of the reported aggregate metrics, not per-contact errors.

MuJoCo: substantially greater compute. The tested configurations cost 18.7–48.8 times as much. The stiff 480 Hz configuration achieves MuJoCo’s smallest maximum overlap, 0.241 mm, still 17.7 times Wrench Transport’s, and retains motion above the settling threshold. The configurations span different controls for compliance, tolerance, and iterative accuracy [3, 4]; no matched-overlap compute ratio is available because none reaches the Wrench Transport 60 Hz result.

At 120 Hz, Wrench Transport follows a different packing trajectory and reaches a slightly larger maximum overlap, still below 0.025 mm; every scene settles. A separate seven-seed native study, with seven measured repetitions after warm-up, records a mean step time of 0.7184 ms, including 2.7550 ms for moving frames and 0.2631 ms for quiet but awake frames. Timing excludes shape cooking, serialization, and rendering.

5.1 Mechanical and numerical checks

The native acceptance suite includes resting rocks, tall stacks, large mass ratios, overhangs, friction inclines, drops, crane holds and releases, support removal, wake propagation, compound bodies, restitution, and deterministic replay. Rest tests span round, flat, and jagged hulls over 50 seeds and 600 simulated seconds per shape class. Tall-stack tests use 15 bodies over 20 seeds; compound-stack tests use ten seeds.

An independent dense Gaussian-elimination comparison checks 80 coupled positive-definite systems against the block solver. The maximum equation residual is 2.398×10^{-14} and maximum solution difference is 3.575×10^{-14} . A separate exhaustive SAT oracle checks 6,000 near-contact convex pairs, with zero missed overlaps, false contacts beyond the margin, or invalid normals in that sample.

6 Discussion

The engine’s practical result is tight, settled contact at a cost in the same range as Rapier and roughly one-twentieth to one-fiftieth of the tested MuJoCo configurations. Default Rapier remains cheaper when its larger overlap is acceptable. Wrench Transport instead reaches sustained rest with sub-0.014 mm maximum final overlap at 60 Hz. Retained impulses preserve support, carried-load scaling keeps the regularization responsive to the supported structure, and sparse elimination communicates the coupled response through the graph.

The three comparison seeds sample a small family of dense packing problems. They do not cover articulated robotics, large open worlds, or sustained high-speed impacts. The implementation currently uses discrete collision with speculative contacts. Continuous collision detection, general joint systems, and deformable bodies are outside its present feature set. Broad-phase enumeration and symbolic adjacency use quadratic structures; dense contact graphs can also increase factorization fill. Those properties define useful directions for larger-scale and high-impact workloads. The inner problem is strongly convex for fixed retained impulses and targets; the outer friction-shift update has a finite pass budget, so its convergence is assessed empirically here.

7 Availability

The C++20 source, CMake package, mechanical and numerical tests, scene generator, comparison runners, and per-run records are available in the BFFT repository. The library exports the `Wrench::Physics` build target and the `zc::phys` C++ interface. A minimal example constructs a body, advances the world, and checks its settled state.

https://github.com/falseywinchnet/bfft/tree/codex/wrench-engine-study/experiments/wrench_transport

References

- [1] A. Castro, F. Permenter, and X. Han. *An Unconstrained Convex Formulation of Compliant Contact*. IEEE Transactions on Robotics, 2022. DOI: 10.1109/TRO.2022.3209077. <https://arxiv.org/abs/2110.10107>.
- [2] D. Baraff. *Linear-Time Dynamics Using Lagrange Multipliers*. SIGGRAPH, 1996, pp. 137–146. <https://www.cs.cmu.edu/~baraff/papers/sig96.pdf>.
- [3] MuJoCo 3.2.3 documentation. *Computation: soft contact model and solvers*. <https://mujoco.readthedocs.io/en/3.2.3/computation/>.
- [4] Dimforge. *Rapier integration parameters*. https://rapier.rs/docs/user_guides/javascript/integration_parameters/.